

Pointers, Ownership, and Object Lifetime in C++

A systems-oriented guide from fundamentals to advanced practice

Contents

1	Why pointers must become second nature	5
2	The memory model you should have in your head	5
3	Pointer declaration syntax	6
3.1	Basic pointers	6
3.2	The declaration trap	6
4	Address-of and dereference	6
4.1	Address-of operator	6
4.2	Dereference operator	6
5	Null pointers	7
6	Stack lifetime versus dynamic lifetime	7
6.1	Automatic storage duration	7
6.2	Dynamic storage duration	7
7	Memory leaks	8
8	Dangling pointers	8
9	Double deletion	9
10	The three concepts: pointer, lifetime, ownership	9
11	RAII: the foundation of modern C++ resource management	9
12	std::unique_ptr	10
12.1	Meaning	10
12.2	Why two unique pointers cannot own the same object	10
12.3	Copying is disabled	10
12.4	Moving transfers ownership	10
12.5	get, release, and reset	10
13	std::shared_ptr	11
13.1	Meaning	11
13.2	Shared ownership has a cost	11
14	std::weak_ptr and ownership cycles	11
15	References versus pointers	12

16 Const correctness with pointers	12
16.1 Pointer to const	12
16.2 Const pointer	12
16.3 Const pointer to const	13
16.4 Reading declarations	13
17 Pointers and arrays	13
17.1 Pointer arithmetic is scaled by the pointed-to type	13
18 The legal domain of pointer arithmetic	13
19 Array decay and function parameters	14
20 Pointers to pointers	14
21 void pointers	15
22 new and delete in detail	15
22.1 Single object	15
22.2 Dynamic array	15
23 new/delete versus malloc/free	15
24 Object lifetime is not the same as storage lifetime	16
25 Placement new	16
26 Alignment	17
27 Pointer casts	17
27.1 static_cast	17
27.2 const_cast	17
27.3 reinterpret_cast	17
27.4 dynamic_cast	17
28 Strict aliasing and type-based access	18
29 Character and byte access	18
30 Function pointers	18
31 Pointers to members	19
32 The arrow operator	19
33 Pointer invalidation	19
34 Pointers into strings and temporary objects	20
35 Raw pointers are not inherently bad	20
36 Borrowing and observer lifetimes	20
37 Returning pointers and references	21
38 Passing pointers to functions	21
38.1 Read-only optional object	21
38.2 Mutable optional object	21

38.3 Required read-only object	21
38.4 Ownership transfer	21
39 Smart pointer function parameters	22
40 Custom deleters	22
41 Incomplete types and unique_ptr	22
42 Pointer stability and data-structure choice	23
43 Memory locality and pointer chasing	23
44 Pointers in networking code	23
45 Pointers in DMA and FPGA host code	24
46 Memory-mapped I/O and volatile	24
47 Pointers and polymorphism	24
48 The Rule of Zero, Three, and Five	24
49 Deep copy versus shallow copy	25
50 Move semantics and ownership	25
51 Exception safety	25
52 Ownership graphs	26
53 Pointer comparison	26
54 Pointer subtraction	26
55 sizeof(pointer) versus sizeof(object)	27
56 Pointer precedence traps	27
57 Common undefined behaviors involving pointers	27
58 Sanitizers	28
59 Valgrind and debugger use	28
60 Concurrency and pointer lifetime	28
61 Atomic shared pointers and lock-free systems	29
62 Pointer provenance and low-level integer conversions	29
63 std::span as a better pointer-plus-length interface	29
64 std::string_view and lifetime	29
65 When to use each ownership tool	30
66 The ownership questions to ask during code review	30

67 Common bad patterns and their replacements	30
67.1 Raw owning pointer	30
67.2 Dynamic array with new[]	31
67.3 Passing pointer and length independently	31
67.4 shared_ptr everywhere	31
68 A systems-programmer mental model	31
69 Advanced example: building a linked list safely	31
70 Advanced example: observer into owned storage	32
71 Advanced example: ownership transfer into a component	32
72 Advanced example: custom resource ownership	32
73 Performance considerations	33
73.1 unique_ptr	33
73.2 shared_ptr	33
73.3 Allocation itself	33
74 Pointer-heavy code and cache behavior	33
75 C interoperability	34
76 What should become automatic	34
77 Practice drills	35
77.1 Drill 1: identify ownership	35
77.2 Drill 2: predict lifetime	35
77.3 Drill 3: predict invalidation	35
77.4 Drill 4: rewrite raw ownership	35
77.5 Drill 5: deliberately trigger sanitizers	35
78 Mini interview questions	35
79 A progression for mastering pointers	36
80 Final rules of thumb	37
81 One-page pointer checklist	38
82 Closing perspective	38

1 Why pointers must become second nature

Pointers are not merely a C or C++ syntax feature. They sit underneath most systems-level abstractions: linked data structures, DMA buffers, network packets, memory-mapped registers, operating-system interfaces, allocators, polymorphism, iterators, smart pointers, and many standard-library types.

A strong C++ programmer should be able to answer the following questions almost automatically whenever a pointer appears:

- What object does this pointer refer to?
- Who owns that object?
- How long is the object guaranteed to remain alive?
- Can this pointer be null?
- Can the pointed-to object be modified through this pointer?
- Is pointer arithmetic valid here?
- Could another pointer refer to the same object?
- What operation eventually destroys the object?
- Is the pointer itself stable, or can reallocation invalidate it?

The central idea of this document is:

The most important pointer rule

A pointer is only useful while the object it points to is alive, and memory ownership must always be unambiguous.

2 The memory model you should have in your head

Consider:

```
1 int x = 42;
2 int* p = &x;
```

Conceptually:

Stack memory

address 0x1000	address 0x2000
+-----+	+-----+
x = 42	p = 0x1000
+-----+	+-----+
^	

The variable `x` stores an integer. The variable `p` stores an address.

The expressions mean:

Expression	Meaning
<code>x</code>	the integer value stored in <code>x</code>
<code>&x</code>	the address of <code>x</code>
<code>p</code>	the address stored in <code>p</code>
<code>*p</code>	the object located at that address

Therefore:

```
1 *p = 100;
```

changes `x` itself.

3 Pointer declaration syntax

3.1 Basic pointers

```
1 int value = 10;
2 int* p = &value;
```

The type of `p` is `int*`: pointer to `int`.

A useful reading habit is to read declarations from the variable outward:

```
1 int* p;
```

“`p` is a pointer to an integer.”

3.2 The declaration trap

```
1 int* a, b;
```

Only `a` is a pointer. `b` is an ordinary `int`.

Prefer:

```
1 int* a;
2 int* b;
```

for clarity.

4 Address-of and dereference

4.1 Address-of operator

```
1 int x = 7;
2 int* p = &x;
```

`&x` obtains the address of `x`.

4.2 Dereference operator

```
1 int y = *p;
```

Dereferencing accesses the object to which the pointer refers.

Dereferencing is a promise

When you write `*p`, you are effectively promising the compiler that `p` points to a live object of a compatible type. Violating that promise leads to undefined behavior.

5 Null pointers

A pointer does not always have to refer to an object.

```
1 int* p = nullptr;
```

`nullptr` is the correct modern C++ null pointer literal.

Do not prefer:

```
1 int* p = 0;
2 int* q = NULL;
```

because `nullptr` has a dedicated pointer-like type and participates in overload resolution correctly.

Before dereferencing a pointer that may be null:

```
1 if (p != nullptr) {
2     std::cout << *p << '\n';
3 }
```

or simply:

```
1 if (p) {
2     std::cout << *p << '\n';
3 }
```

6 Stack lifetime versus dynamic lifetime

6.1 Automatic storage duration

```
1 void f()
2 {
3     int x = 10;
4 }
```

`x` is destroyed automatically when control leaves the block.

6.2 Dynamic storage duration

```
1 int* p = new int(10);
```

The integer lives until it is explicitly destroyed:

```
1 delete p;
```

This creates a major responsibility: the programmer must ensure exactly one matching destruction occurs.

Manual allocation is usually not the best C++ interface

Modern C++ generally avoids naked `new` and `delete` in application code. Prefer automatic objects, containers, and RAII wrappers.

7 Memory leaks

A memory leak occurs when dynamically allocated memory remains alive but no usable pointer to it remains.

```
1 void leak()
2 {
3     int* p = new int(10);
4     p = nullptr;
5 }
```

The allocated integer still exists, but its address has been lost.

A subtler leak:

```
1 int* p = new int(10);
2 p = new int(20);
```

The first allocation is leaked when `p` is overwritten.

8 Dangling pointers

A dangling pointer contains the address of an object that no longer exists.

```
1 int* p = new int(10);
2 delete p;
3 std::cout << *p << '\n';
```

The call to `delete` destroys the object, but it does not magically rewrite every pointer containing that address.

Setting a local pointer to null afterward can help prevent accidental reuse:

```
1 delete p;
2 p = nullptr;
```

However, this does not solve aliasing:

```
1 int* p = new int(10);
2 int* q = p;
3
4 delete p;
5 p = nullptr;
6
7 // q is still dangling.
```


9 Double deletion

Consider:

```
1  int* p = new int(10);
2
3  delete p;
4  delete p;
```

The second deletion is undefined behavior.

The same fundamental mistake can appear through aliases:

```
1  int* p = new int(10);
2  int* q = p;
3
4  delete p;
5  delete q;
```

This is exactly why ownership must be explicit.

10 The three concepts: pointer, lifetime, ownership

These concepts are related but not identical.

1. **Pointer:** stores an address.
2. **Lifetime:** the period during which the object exists.
3. **Ownership:** responsibility for eventually destroying the object.

Multiple pointers may point to the same object while only one abstraction owns it.

```
1  auto owner = std::make_unique<int>(10);
2  int* observer = owner.get();
```

Both names access the same integer, but only **owner** is responsible for its destruction.

11 RAI: the foundation of modern C++ resource management

RAII means *Resource Acquisition Is Initialization*. A resource is bound to the lifetime of an object whose destructor releases it.

The resource might be:

- heap memory,
- a file descriptor,
- a socket,
- a mutex lock,
- a DMA buffer,
- a memory mapping,
- a hardware handle.

Instead of manually remembering cleanup paths, RAI makes normal language scope perform cleanup automatically.

12 std::unique_ptr

12.1 Meaning

A `std::unique_ptr<T>` represents exclusive ownership of a dynamically allocated T.

```
1 auto p = std::make_unique<int>(10);
```

When `p` leaves scope, the integer is destroyed automatically.

12.2 Why two unique pointers cannot own the same object

This is wrong:

```
1 auto* raw = new int(10);
2
3 std::unique_ptr<int> p1(raw);
4 std::unique_ptr<int> p2(raw);
```

Both smart pointers believe they independently own the same integer. Both will attempt to destroy it. This is a double deletion bug.

12.3 Copying is disabled

```
1 auto p1 = std::make_unique<int>(10);
2 std::unique_ptr<int> p2 = p1;
```

This does not compile. Copying would imply two owners.

12.4 Moving transfers ownership

```
1 auto p1 = std::make_unique<int>(10);
2 auto p2 = std::move(p1);
```

After the move:

`p1 ---> nullptr`

`p2 ---> [10]`

Ownership has moved; it has not been duplicated.

12.5 get, release, and reset

`get()` returns the raw pointer without changing ownership:

```
1 auto p = std::make_unique<int>(10);
2 int* raw = p.get();
```

`release()` gives up ownership and returns the raw pointer:

```
1 int* raw = p.release();
```

The caller must now arrange destruction. This operation is dangerous and should be used only when ownership transfer to a raw-pointer API is genuinely required.

`reset()` destroys the currently owned object and optionally takes ownership of a new one:

```
1 p.reset();
2 p.reset(new int(20));
```

Prefer `make_unique` when initially constructing the object.

13 `std::shared_ptr`

13.1 Meaning

A `std::shared_ptr<T>` represents shared ownership.

```
1 auto p1 = std::make_shared<int>(10);
2 auto p2 = p1;
```

Conceptually, the smart pointers share a control block containing ownership metadata, including a reference count.

```
p1 ----+
      |
      +-----+
p2 ----+-----> | ctrl | ----> | int = 10 |
                  +-----+
                  count = 2
```

The object is destroyed when the final owning `shared_ptr` disappears.

13.2 Shared ownership has a cost

Compared with `unique_ptr`, `shared_ptr` generally requires:

- a control block,
- reference-count updates,
- usually atomic operations on that reference count,
- more complicated lifetime reasoning.

Do not use `shared_ptr` merely because it feels safer. Use it when the ownership model is genuinely shared.

14 `std::weak_ptr` and ownership cycles

Two objects owning each other through `shared_ptr` can form a cycle.

```
1 struct Node {
2     std::shared_ptr<Node> next;
3 };

```

More complex mutually owning graphs can leave reference counts permanently above zero.

A `weak_ptr` observes an object managed by `shared_ptr` without contributing to its ownership count.

```
1 std::weak_ptr<Node> parent;
```

To access the object safely:

```

1  if (auto p = parent.lock()) {
2      // p is a shared_ptr and keeps the object alive here.
3  }

```

15 References versus pointers

A reference is an alias to an existing object.

```

1  int x = 10;
2  int& r = x;

```

Important differences:

Property	Reference	Pointer
Can represent no object?	Normally no	Yes, via <code>nullptr</code>
Can be reseated?	No	Yes
Needs explicit dereference?	No	Yes
Supports pointer arithmetic?	No	Yes
Typical API meaning	Required object	Optional/indirect object

A useful interface rule is:

Reference versus pointer API rule

Use a reference when an object is required. Use a pointer when “no object” is a meaningful state or pointer semantics are actually required.

16 Const correctness with pointers

This is one of the most important pointer topics to become fluent in.

16.1 Pointer to const

```

1  const int* p = &x;

```

or equivalently:

```

1  int const* p = &x;

```

You may change `p`, but you may not modify the integer through `p`.

```

1  p = &y;    // allowed
2  *p = 5;    // not allowed

```

16.2 Const pointer

```

1  int* const p = &x;

```

The pointer itself cannot be changed, but the object can.

```

1  *p = 5;    // allowed
2  p = &y;    // not allowed

```

16.3 Const pointer to const

```
1  const int* const p = &x;
```

Neither the pointer nor the pointed-to object may be changed through this expression.

16.4 Reading declarations

A useful rule is to read from the identifier outward:

```
1  const int* p;  
2  int* const p2 = &x;  
3  const int* const p3 = &x;
```

- p: pointer to const int.
- p2: const pointer to int.
- p3: const pointer to const int.

17 Pointers and arrays

Arrays and pointers are closely related but are not the same thing.

```
1  int a[4] = {10, 20, 30, 40};  
2  int* p = a;
```

In many expressions, the array name decays to a pointer to its first element.

Therefore:

```
1  a[2]  
2  *(a + 2)  
3  p[2]  
4  *(p + 2)
```

all access the same element.

17.1 Pointer arithmetic is scaled by the pointed-to type

If p is an `int*`, then:

```
1  p + 1
```

means “advance by one `int`,” not “advance by one byte.”

The actual byte increment is `sizeof(int)`.

18 The legal domain of pointer arithmetic

Pointer arithmetic is only defined within a single array object, plus the special one-past-the-end position.

```
1  int a[4];  
2  int* begin = &a[0];  
3  int* end = &a[4];
```

`end` may be formed and compared, but must not be dereferenced.

Pointer arithmetic is not arbitrary integer arithmetic

Adding offsets to unrelated addresses and dereferencing the result is not a portable memory-access mechanism in C++.

19 Array decay and function parameters

Consider:

```
1 void f(int arr[])
2 {
3 }
```

In a function parameter, this is effectively:

```
1 void f(int* arr)
2 {
3 }
```

The function does not know the array length from the pointer alone.

Modern C++ often uses:

```
1 void f(std::span<const int> values);
```

because `std::span` carries both a pointer and a length while remaining a lightweight non-owning view.

20 Pointers to pointers

A pointer can point to another pointer.

```
1 int x = 10;
2 int* p = &x;
3 int** pp = &p;
```

Then:

```
1 *p    // x
2 **pp  // x
```

This is common when a function needs to modify the caller's pointer itself.

C-style example:

```
1 void allocate(int** out)
2 {
3     *out = new int(10);
4 }
```

Modern C++ would normally redesign this interface to return an owning object:

```
1 std::unique_ptr<int> allocate()
2 {
3     return std::make_unique<int>(10);
```

```
4 }
```

21 void pointers

A `void*` can hold the address of an object without encoding its concrete type.

```
1 int x = 10;
2 void* raw = &x;
```

You cannot directly dereference a `void*`. It must first be converted to an appropriate typed pointer.

```
1 int* p = static_cast<int*>(raw);
```

`void*` is common in C APIs, operating-system interfaces, allocators, callbacks, and low-level libraries. Modern C++ interfaces usually preserve stronger type information where possible.

22 new and delete in detail

22.1 Single object

```
1 int* p = new int(10);
2 delete p;
```

22.2 Dynamic array

```
1 int* p = new int[100];
2 delete[] p;
```

The forms must match.

Never mismatch allocation and deallocation forms

`new` pairs with `delete`. `new[]` pairs with `delete[]`. `malloc` pairs with `free`.

For ordinary dynamic arrays, prefer `std::vector`.

23 new/delete versus malloc/free

C allocation:

```
1 void* mem = std::malloc(sizeof(MyType));
2 std::free(mem);
```

C++ allocation:

```
1 MyType* p = new MyType(args);
2 delete p;
```

The conceptual difference matters:

- `malloc` obtains raw storage.
- `new` obtains storage and begins object lifetime by construction.

- `delete` runs the destructor and releases storage.
- `free` merely releases compatible C-allocated storage.

Never write:

```
1  int* p = new int(10);
2  std::free(p);
```

or:

```
1  void* p = std::malloc(100);
2  delete p;
```

24 Object lifetime is not the same as storage lifetime

This distinction becomes important in allocators, unions, placement new, and high-performance systems code.

Memory may exist before an object exists in it, and memory may remain after an object's lifetime has ended.

A simplified conceptual sequence is:

1. Obtain suitably sized and aligned storage.
2. Construct an object in that storage.
3. Use the object while its lifetime is active.
4. Destroy the object.
5. Release or reuse the storage.

Most application code should use abstractions that manage these stages. Advanced systems code sometimes manipulates them explicitly.

25 Placement new

Placement new constructs an object at a caller-supplied address.

```
1  #include <new>
2
3  alignas(MyType) unsigned char storage[sizeof(MyType)];
4
5  MyType* p = new (storage) MyType(args);
6
7  p->~MyType();
```

Placement new does not allocate memory. It begins an object's lifetime in existing storage.

This technique appears in:

- allocators,
- object pools,
- embedded systems,
- shared-memory layouts,

- lock-free containers,
- high-performance memory arenas.

It should not be used casually.

26 Alignment

Objects have alignment requirements.

```
1 alignof(std::uint64_t)
```

A pointer may contain a numerically valid address but still be invalid for a type if the address is not suitably aligned.

Low-level code dealing with DMA buffers, SIMD vectors, packed protocols, or memory-mapped structures must be especially careful about alignment.

27 Pointer casts

C++ provides several explicit cast forms because different conversions have very different meanings.

27.1 static_cast

Use for conversions that are part of the language's normal type relationships.

```
1 void* raw = &x;  
2 int* p = static_cast<int*>(raw);
```

27.2 const_cast

Changes cv-qualification.

```
1 const int* p = &x;  
2 int* q = const_cast<int*>(p);
```

Casting away constness does not make a genuinely const object modifiable. Writing through such a pointer can be undefined behavior.

27.3 reinterpret_cast

Requests a low-level reinterpretation of representation or pointer type.

```
1 auto addr = reinterpret_cast<std::uintptr_t>(p);
```

This is a sharp tool. The fact that a conversion compiles does not mean the result may legally be dereferenced.

27.4 dynamic_cast

Used with polymorphic class hierarchies for checked runtime conversions.

```
1 Base* base = get_object();  
2 Derived* derived = dynamic_cast<Derived*>(base);  
3  
4 if (derived) {
```

```
5 // conversion succeeded
6 }
```

28 Strict aliasing and type-based access

Optimizing compilers reason about which pointer types can legally refer to the same object. Violating those assumptions can create bugs that appear only under optimization.

Dangerous style:

```
1 float f = 1.0f;
2 int* bits = reinterpret_cast<int*>(&f);
3 int x = *bits;
```

For representation conversion, modern C++ often provides safer mechanisms such as `std::bit_cast` when the requirements are met.

```
1 #include <bit>
2
3 std::uint32_t bits = std::bit_cast<std::uint32_t>(f);
```

29 Character and byte access

Object representations may be examined through character-like byte types. Modern code commonly uses `std::byte` for raw byte-oriented work.

```
1 std::byte* bytes = reinterpret_cast<std::byte*>(&object);
```

This matters in serializers, packet parsers, binary protocols, and hardware interfaces.

30 Function pointers

Functions have addresses too.

```
1 int add(int a, int b)
2 {
3     return a + b;
4 }
5
6 int (*fn)(int, int) = &add;
7 int result = fn(2, 3);
```

Function pointers are common in:

- callback APIs,
- interrupt tables,
- dispatch tables,
- C interoperability,
- low-level state machines.

Modern generic C++ may instead use lambdas, templates, or `std::function` depending on the performance and ownership requirements.

31 Pointers to members

C++ also supports pointers to members, which are not ordinary memory addresses.

```
1 struct Packet {  
2     int price;  
3 };  
4  
5 int Packet::* member = &Packet::price;  
6  
7 Packet p{100};  
8 int value = p.*member;
```

With a pointer to an object:

```
1 Packet* ptr = &p;  
2 int value = ptr->*member;
```

This is more specialized but useful to recognize.

32 The arrow operator

For a pointer to an object:

```
1 Node* p = ...;  
2 p->next
```

is equivalent to:

```
1 (*p).next
```

Parentheses are required in the second form because member access has higher precedence than unary dereference.

33 Pointer invalidation

A pointer can become invalid even when no explicit `delete` appears. Container operations may relocate elements.

```
1 std::vector<int> v;  
2 v.push_back(10);  
3  
4 int* p = &v[0];  
5  
6 v.push_back(20);  
7 v.push_back(30);
```

If the vector reallocates, `p` may become dangling.

The same concern applies to:

- pointers,
- references,
- iterators,

- spans,
- string views.

An advanced programmer learns the invalidation guarantees of the containers being used.

34 Pointers into strings and temporary objects

This is dangerous:

```
1  const char* get_name()
2  {
3      std::string s = "market";
4      return s.c_str();
5  }
```

The returned pointer refers to storage owned by `s`, which is destroyed when the function returns.

Similarly, non-owning views such as `std::string_view` require the underlying character sequence to outlive the view.

35 Raw pointers are not inherently bad

A raw pointer is perfectly reasonable when it is explicitly non-owning.

```
1  void process(const Packet* packet)
2  {
3      if (!packet) {
4          return;
5      }
6
7      // Observe packet. Do not delete it.
8  }
```

The dangerous situation is not “raw pointer exists.” It is “ownership and lifetime are unclear.”

Modern C++ design frequently follows this convention:

- owning dynamic object: `unique_ptr` or `shared_ptr`,
- required non-owning object: reference,
- optional non-owning object: raw pointer,
- contiguous non-owning range: `span`,
- non-owning string range: `string_view`.

36 Borrowing and observer lifetimes

Consider:

```
1  auto owner = std::make_unique<Node>();
2  Node* observer = owner.get();
```

This is valid while `owner` still owns the node.

But:

```
1 owner.reset();
2 observer->value = 5;
```

is a use-after-free.

Whenever storing a non-owning pointer, ask:

What guarantees that the owner outlives this observer?

That question is central in large C++ systems.

37 Returning pointers and references

Never return a pointer or reference to a local automatic variable.

```
1 int* bad()
2 {
3     int x = 10;
4     return &x;
5 }
```

`x` dies when the function returns.

Returning a pointer can be valid when the pointee outlives the call:

```
1 Node* find_node(Node& root, int key);
```

But the interface must make the lifetime assumptions understandable.

38 Passing pointers to functions

38.1 Read-only optional object

```
1 void inspect(const Packet* packet);
```

38.2 Mutable optional object

```
1 void modify(Packet* packet);
```

38.3 Required read-only object

Prefer:

```
1 void inspect(const Packet& packet);
```

38.4 Ownership transfer

Make it explicit:

```
1 void install(std::unique_ptr<Device> device);
```

The caller can see immediately that ownership moves into the function.

39 Smart pointer function parameters

Avoid passing smart pointers when the function only needs to access the object.

Instead of:

```
1 void process(const std::unique_ptr<Packet>& packet);
```

prefer:

```
1 void process(const Packet& packet);
```

unless the function genuinely needs ownership semantics.

Use a smart-pointer parameter when ownership itself is part of the API contract.

40 Custom deleters

`unique_ptr` can manage resources that require something other than ordinary `delete`.

Example with a C file handle:

```
1 #include <cstdio>
2 #include <memory>
3
4 struct FileCloser {
5     void operator()(std::FILE* file) const
6     {
7         if (file) {
8             std::fclose(file);
9         }
10    }
11 };
12
13 using FilePtr = std::unique_ptr<std::FILE, FileCloser>;
```

Now the file automatically closes when the wrapper leaves scope.

The same pattern works for sockets, mappings, hardware handles, and library resources.

41 Incomplete types and `unique_ptr`

`unique_ptr` is useful for the PImpl idiom because a class can hold a pointer to an implementation type whose definition is hidden from the header.

```
1 class Engine {
2 public:
3     Engine();
4     ~Engine();
5
6 private:
7     struct Impl;
8     std::unique_ptr<Impl> impl_;
9 };
```

The destructor is commonly defined in the source file after `Impl` is complete.

This reduces header dependencies and can improve compile times and ABI hygiene.

42 Pointer stability and data-structure choice

If code stores addresses to elements, the data structure's relocation behavior matters.

For example, vector growth may invalidate element pointers. A node-based container often provides different stability properties, though at the cost of additional indirection and worse locality.

An advanced programmer does not ask only:

Which container has the operation I need?

They also ask:

What are its allocation, locality, iterator, pointer, and reference stability properties?

43 Memory locality and pointer chasing

Pointers have performance implications.

A linked list:

```
[node] -> [node] -> [node] -> [node]
```

may scatter nodes across memory. Traversal can cause cache misses because each node tells the CPU where to load next.

A vector:

```
[element][element][element][element]
```

stores elements contiguously, which is usually friendlier to caches and hardware prefetchers.

This matters heavily in low-latency systems. Reducing pointer chasing is often more important than reducing the theoretical number of arithmetic operations.

44 Pointers in networking code

Low-level networking often exposes raw byte buffers.

```
1  std::array<std::byte, 2048> buffer;  
2  
3  ssize_t n = recv(fd,  
4              buffer.data(),  
5              buffer.size(),  
6              0);
```

The pointer returned by `buffer.data()` does not own memory. The `std::array` owns the storage.

When parsing packet fields, be careful about:

- buffer bounds,
- alignment,
- byte order,
- type aliasing,
- lifetime,
- untrusted lengths.

45 Pointers in DMA and FPGA host code

DMA programming frequently involves a virtual address on the CPU side and a physical or device-visible address for hardware.

A pointer such as:

```
1 std::uint32_t* words;
```

is a CPU-language view into memory. It does not by itself tell you:

- whether the memory is physically contiguous,
- whether it is DMA-capable,
- whether caches are coherent with the device,
- what physical address hardware sees,
- whether synchronization is required.

This distinction is important when moving from normal C++ memory to FPGA or NIC-facing buffers.

46 Memory-mapped I/O and volatile

Hardware registers are often accessed through memory-mapped addresses.

Conceptually:

```
1 volatile std::uint32_t* status = ...;
2 std::uint32_t value = *status;
```

`volatile` tells the compiler that accesses are externally observable and must not be optimized away in the ordinary way.

However, `volatile` is not a general synchronization primitive and does not replace atomics or memory-ordering mechanisms for inter-thread communication.

47 Pointers and polymorphism

A base-class pointer can refer to a derived object.

```
1 struct Base {
2     virtual ~Base() = default;
3 };
4
5 struct Derived : Base {
6 };
7
8 std::unique_ptr<Base> p = std::make_unique<Derived>();
```

The virtual destructor matters when deleting through the base type.

Without a suitable virtual destructor, deleting a derived object through a base pointer can cause undefined behavior.

48 The Rule of Zero, Three, and Five

A class manually owning a raw resource must carefully define copy, move, and destruction behavior.

Historically, this leads to the Rule of Three and Rule of Five.

Modern C++ aims for the Rule of Zero: store resources in RAII types so that your class needs no custom destructor, copy constructor, move constructor, copy assignment operator, or move assignment operator.

Example:

```
1 class Buffer {
2 private:
3     std::vector<std::byte> data_;
4 };
```

The vector handles ownership correctly.

49 Deep copy versus shallow copy

Suppose a class owns a raw pointer:

```
1 struct BadBuffer {
2     std::size_t size;
3     int* data;
4 };
```

The compiler-generated copy operation copies only the address:

```
A.data ----+
           +----> same allocation
B.data ----+
```

If both objects think they own that allocation, destruction can double-delete it.

This is a classic shallow-copy ownership bug.

A deep copy creates a separate allocation and copies the actual contents.

Better yet, use `std::vector<int>` unless manual ownership is required.

50 Move semantics and ownership

Move semantics are fundamental to modern resource ownership.

A type such as `unique_ptr` can be moved because ownership can be transferred safely.

```
1 std::unique_ptr<int> make_value()
2 {
3     return std::make_unique<int>(42);
4 }
```

Returning the pointer transfers ownership efficiently without duplicating the resource.

This pattern makes explicit ownership transfer a first-class language concept.

51 Exception safety

Manual resource management becomes especially dangerous when exceptions can interrupt control flow.

Fragile code:

```
1 Widget* w = new Widget;
2 perform_operation();
3 delete w;
```

If `perform_operation()` throws, `delete` may never execute.

RAII fixes this:

```
1 auto w = std::make_unique<Widget>();
2 perform_operation();
```

The destructor runs during stack unwinding.

52 Ownership graphs

For larger software systems, it is useful to think of ownership as a graph.

A healthy design often resembles a tree:

```
Engine
|
+-- Parser
|
+-- Planner
|
+-- Executor
    |
    +-- Buffer
```

Strong ownership points downward. Non-owning observers can point elsewhere.

Cycles of strong ownership should make you suspicious and usually require a clear reason.

53 Pointer comparison

Pointers can be tested for equality:

```
1 if (p == q) {
2     // same address
3 }
```

Relational pointer comparisons are only meaningful under specific conditions, most notably for pointers into the same array object. Do not treat arbitrary object pointers as portable sortable integers.

54 Pointer subtraction

For two pointers into the same array:

```
1 std::ptrdiff_t distance = end - begin;
```

The result measures the number of elements, not bytes.

This is the basis of many iterator and range operations.

55 sizeof(pointer) versus sizeof(object)

```
1  int* p = nullptr;
2
3  sizeof(p)
4  sizeof(*p)
```

`sizeof(p)` is the size of the pointer itself.

`sizeof(*p)` is the size of an `int`. Note that the operand of `sizeof` is generally not evaluated, so this does not dereference a null pointer at runtime.

56 Pointer precedence traps

Examples worth memorizing:

```
1  *p++
```

means:

```
1  *(p++)
```

not:

```
1  (*p)++
```

Similarly:

```
1  ++*p
2  ++*p
```

mean very different things.

In systems code, clarity is more valuable than showing off precedence knowledge. Parenthesize expressions if a reader could hesitate.

57 Common undefined behaviors involving pointers

You should recognize these instantly:

1. Dereferencing `nullptr`.
2. Dereferencing a dangling pointer.
3. Use after `delete` or `free`.
4. Double `delete` or double `free`.
5. Out-of-bounds pointer arithmetic.
6. Dereferencing one-past-the-end.
7. Mismatching `new/delete[]` forms.
8. Mismatching C and C++ allocators.
9. Accessing an object through an incompatible type.
10. Returning a pointer or reference to a dead local object.

11. Using a pointer invalidated by container reallocation.
12. Violating alignment requirements.

58 Sanitizers

Compiler sanitizers are among the best tools for learning pointer correctness.

With GCC or Clang:

```
1 g++ -std=c++20 \  
2     -Wall -Wextra -Wpedantic \  
3     -fsanitize=address,undefined \  
4     -fno-omit-frame-pointer \  
5     program.cpp -o program
```

AddressSanitizer can detect many instances of:

- use-after-free,
- heap buffer overflow,
- stack buffer overflow,
- double free,
- some memory leaks.

UndefinedBehaviorSanitizer catches many additional invalid operations.

59 Valgrind and debugger use

On supported platforms, Valgrind can help locate invalid accesses and leaks.

In a debugger, inspect pointer values and pointees separately:

```
1 p p  
2 p *p  
3 x/16xb p
```

The exact commands vary by debugger, but the mental distinction is the same: address versus object.

60 Concurrency and pointer lifetime

A pointer that is valid in one thread can become dangling if another thread destroys the object without synchronization.

This creates two separate correctness questions:

1. Is the memory still alive?
2. Are accesses properly synchronized?

A `shared_ptr` can help coordinate lifetime, but it does not magically make the pointed-to object thread-safe.

61 Atomic shared pointers and lock-free systems

Advanced concurrent code may require atomic publication of pointer-like objects. Modern C++ provides facilities for atomically manipulating `shared_ptr` and atomic pointer types.

However, lifetime-safe lock-free programming is substantially more difficult than ordinary pointer programming. Techniques such as hazard pointers, epoch reclamation, or read-copy-update exist because reclaiming memory safely while other threads may still hold addresses is hard.

The key lesson is that reclamation, not merely pointer update, is often the hardest part of lock-free design.

62 Pointer provenance and low-level integer conversions

Systems programmers sometimes convert pointers to integer address types:

```
1 std::uintptr_t addr =  
2     reinterpret_cast<std::uintptr_t>(p);
```

This can be useful for logging or low-level interfaces, but portable C++ does not reduce pointer validity to “the integer happens to contain the same numeric address.”

Modern C++ object and pointer rules are stricter than a simple flat-address machine model. Be cautious when manufacturing pointers from arbitrary integers.

63 `std::span` as a better pointer-plus-length interface

Traditional C APIs commonly use:

```
1 void process(const std::uint32_t* data,  
2             std::size_t count);
```

Modern C++ can often use:

```
1 void process(std::span<const std::uint32_t> data);
```

This makes the pair inseparable at the type level and reduces accidental length mismatches while remaining non-owning and lightweight.

64 `std::string_view` and lifetime

`std::string_view` is pointer-like: it does not own the string data.

Safe:

```
1 std::string name = "engine";  
2 std::string_view view = name;
```

Potentially dangerous:

```
1 std::string_view make_view()  
2 {  
3     std::string s = "temporary";  
4     return s;  
5 }
```

The returned view refers to destroyed storage.

65 When to use each ownership tool

Tool	Typical meaning
Local object	Best default when dynamic lifetime is unnecessary.
<code>std::vector<T></code>	Owning dynamic contiguous sequence.
<code>std::string</code>	Owning dynamic character sequence.
<code>std::unique_ptr<T></code>	Exclusive dynamic ownership.
<code>std::shared_ptr<T></code>	Genuine shared ownership.
<code>std::weak_ptr<T></code>	Non-owning observation of shared ownership.
<code>T&</code>	Required non-owning access.
<code>const T&</code>	Required read-only non-owning access.
<code>T*</code>	Optional or pointer-semantic non-owning access.
<code>std::span<T></code>	Non-owning contiguous range.
<code>std::string_view</code>	Non-owning character range.

66 The ownership questions to ask during code review

Whenever you encounter a pointer, ask:

1. Who created this object?
 2. Who destroys it?
 3. Can the pointer be null?
 4. Is this pointer owning or non-owning?
 5. What keeps the pointee alive?
 6. Can reallocation invalidate it?
 7. Can another thread destroy it?
 8. Is the pointee mutable through this interface?
 9. Are there multiple owners?
 10. Is the ownership transfer visible in the type system?
- If these questions are hard to answer, the interface probably needs redesign.

67 Common bad patterns and their replacements

67.1 Raw owning pointer

Avoid:

```
1 Widget* widget = new Widget;
```

Prefer:

```
1 auto widget = std::make_unique<Widget>();
```

67.2 Dynamic array with new[]

Avoid:

```
1 int* values = new int[n];
```

Prefer:

```
1 std::vector<int> values(n);
```

67.3 Passing pointer and length independently

Prefer a range abstraction where appropriate:

```
1 void process(std::span<const int> values);
```

67.4 shared_ptr everywhere

Avoid using shared ownership by default. Prefer exclusive ownership until the architecture genuinely requires shared lifetime.

68 A systems-programmer mental model

For every pointer-like expression, classify it along four axes:

Axis	Question	Example A	Example B
Ownership	Who destroys?	owner	observer
Nullability	Can it be empty?	pointer	reference
Mutability	Can pointee change?	T*	const T*
Extent	One or many objects?	T*	span<T>

This classification prevents many design mistakes before code is written.

69 Advanced example: building a linked list safely

A raw-pointer implementation often requires custom destruction logic. Exclusive ownership can instead be encoded directly into the node structure.

```
1 struct Node {
2     int value;
3     std::unique_ptr<Node> next;
4 };
5
6 class List {
7 public:
8     void push_front(int value)
9     {
10         auto node = std::make_unique<Node>();
11         node->value = value;
12         node->next = std::move(head_);
13         head_ = std::move(node);
14     }
15
16 private:
17     std::unique_ptr<Node> head_;
18 };
```

When `List` is destroyed, ownership recursively releases the chain.

70 Advanced example: observer into owned storage

```

1  class Book {
2  public:
3      const Order* best_bid() const noexcept
4      {
5          if (orders_.empty()) {
6              return nullptr;
7          }
8
9          return &orders_.front();
10     }
11
12 private:
13     std::vector<Order> orders_;
14 };

```

This interface returns a non-owning pointer. The caller must understand that a future modification of `orders_` may invalidate the returned pointer.

An API that exposes interior pointers is therefore also exposing lifetime and invalidation constraints.

71 Advanced example: ownership transfer into a component

```

1  class Engine {
2  public:
3      void set_backend(std::unique_ptr<Backend> backend)
4      {
5          backend_ = std::move(backend);
6      }
7
8  private:
9      std::unique_ptr<Backend> backend_;
10 };

```

At the call site:

```

1  auto backend = std::make_unique<FpgaBackend>();
2  engine.set_backend(std::move(backend));

```

The type and the explicit move make the ownership transition visible.

72 Advanced example: custom resource ownership

Suppose an operating-system API returns an integer file descriptor.

```

1  class FileDescriptor {
2  public:
3      explicit FileDescriptor(int fd)
4          : fd_(fd)
5      {
6      }
7
8      ~FileDescriptor()

```



```
9      {
10          if (fd_ >= 0) {
11              close(fd_);
12          }
13      }
14
15      FileDescriptor(const FileDescriptor&) = delete;
16      FileDescriptor& operator=(const FileDescriptor&) = delete;
17
18      FileDescriptor(FileDescriptor&& other) noexcept
19          : fd_(other.fd_)
20      {
21          other.fd_ = -1;
22      }
23
24  private:
25      int fd_ = -1;
26  };
```

This demonstrates that smart-pointer thinking is really resource-ownership thinking. Not every resource is literally a pointer.

73 Performance considerations

73.1 `unique_ptr`

A `unique_ptr` is typically designed to be essentially as lightweight as a raw pointer when using a stateless default deleter. Its main benefit is compile-time ownership discipline and automatic destruction.

73.2 `shared_ptr`

A `shared_ptr` normally introduces reference-counting and control-block costs. In low-latency code, this can matter.

73.3 Allocation itself

Heap allocation can be expensive or unpredictable relative to stack allocation or preallocated pools. Latency-sensitive systems frequently use:

- object pools,
- arenas,
- slab allocators,
- ring buffers,
- preallocated vectors,
- fixed-capacity containers.

The goal is often not “avoid pointers,” but “make ownership, locality, and allocation behavior predictable.”

74 Pointer-heavy code and cache behavior

A useful performance hierarchy is:

1. contiguous data is usually easier for hardware to prefetch,
2. fewer dependent pointer dereferences usually means less latency,
3. smaller objects often improve cache utilization,
4. unpredictable heap allocation can hurt deterministic latency.

This is why high-performance code often favors arrays of data over graphs of individually allocated objects.

75 C interoperability

C APIs commonly express ownership only in documentation.

For example:

```
1 resource_t* resource_create();
2 void resource_destroy(resource_t* resource);
```

A C++ wrapper can encode this ownership:

```
1 struct ResourceDeleter {
2     void operator()(resource_t* resource) const
3     {
4         resource_destroy(resource);
5     }
6 };
7
8 using ResourcePtr =
9     std::unique_ptr<resource_t, ResourceDeleter>;
```

This is a powerful pattern for wrapping operating-system and hardware APIs.

76 What should become automatic

You should eventually see each of the following and react immediately:

- `new` → ask who owns the result.
- raw pointer field → ask owning or observing?
- `delete` → ask why RAII is not handling it.
- `shared_ptr` → ask whether ownership is truly shared.
- `std::move(unique_ptr)` → ownership transfer.
- `vector.push_back` → possible reallocation/invalidation.
- returned reference/pointer → ask what outlives what.
- `reinterpret_cast` → inspect alignment, aliasing, lifetime.
- pointer plus offset → verify array bounds and element type.
- `string_view/span` → remember non-ownership.

77 Practice drills

77.1 Drill 1: identify ownership

For every pointer in a program, label it as one of:

- owner,
- observer,
- optional observer,
- shared owner,
- temporary iterator-like address.

77.2 Drill 2: predict lifetime

Before running code, write down exactly when each object is constructed and destroyed.

77.3 Drill 3: predict invalidation

Take a vector program and identify which operations can invalidate pointers, references, and iterators into it.

77.4 Drill 4: rewrite raw ownership

Convert a manually managed linked list from `new/delete` to `unique_ptr` ownership.

77.5 Drill 5: deliberately trigger sanitizers

Write small programs containing:

- a double free,
- a use-after-free,
- a heap out-of-bounds access,
- a stack dangling pointer.

Run them under AddressSanitizer and learn to read the reports.

78 Mini interview questions

You should be able to answer these without hesitation.

1. What is the difference between `int* p` and `int& r`?
2. What is a dangling pointer?
3. What does `std::move` do to a `unique_ptr`?
4. Why can a `shared_ptr` cycle leak memory?
5. What is the difference between `const int*` and `int* const`?
6. When does a vector invalidate pointers to its elements?
7. Why is returning `&local` invalid?
8. Why is `delete[]` different from `delete`?
9. What does `unique_ptr::release()` do?

10. Why does a base class often need a virtual destructor?
11. Why is pointer arithmetic only safe within an array object?
12. What is the distinction between storage lifetime and object lifetime?
13. What is placement new?
14. Why can `reinterpret_cast` code break under optimization?
15. What problem does `std::span` solve?

79 A progression for mastering pointers

Level 1: syntax

Master:

- `&`, `*`, `->`,
- `nullptr`,
- references,
- arrays and pointer arithmetic.

Level 2: lifetime

Master:

- stack versus heap lifetime,
- dangling pointers,
- leaks,
- invalidation,
- temporary-object lifetime.

Level 3: ownership

Master:

- RAII,
- `unique_ptr`,
- move semantics,
- `shared_ptr`,
- `weak_ptr`.

Level 4: systems correctness

Master:

- alignment,
- aliasing,
- object lifetime,
- byte representations,

- allocator boundaries,
- memory-mapped I/O,
- DMA memory assumptions.

Level 5: performance and concurrency

Master:

- pointer chasing and cache misses,
- allocation strategies,
- pointer stability,
- concurrent lifetime management,
- memory reclamation.

80 Final rules of thumb

Rule 1

Prefer values and automatic storage unless dynamic lifetime is actually needed.

Rule 2

If one object owns a dynamic resource, encode that ownership with RAII, usually `std::unique_ptr` or a standard container.

Rule 3

Use `shared_ptr` only when multiple components genuinely share ownership, not merely because multiple components need access.

Rule 4

Treat raw pointers as non-owning unless an API explicitly documents otherwise.

Rule 5

A non-owning pointer is safe only if the pointee is guaranteed to outlive every use of that pointer.

Rule 6

Know which operations invalidate pointers, references, iterators, spans, and views into containers.

Rule 7

When using low-level casts, reason about type, alignment, object lifetime, and aliasing separately. A plausible address is not enough.

Rule 8

Use sanitizers routinely while developing systems code. Pointer bugs are much easier to learn from when the runtime identifies the exact misuse.

81 One-page pointer checklist

Before approving pointer-heavy code, verify:

- pointee is alive,
- nullability is understood,
- owner is known,
- destruction happens once,
- no leaks,
- no dangling observers,
- correct `new/delete` pairing,
- no invalid pointer arithmetic,
- no out-of-bounds dereference,
- constness is intentional,
- container invalidation is understood,
- alignment is valid,
- aliasing is legal,
- ownership transfer is explicit,
- shared ownership is justified,
- concurrency lifetime is synchronized,
- non-owning views do not outlive data,
- casts are justified,
- resource cleanup is RAII-based,
- sanitizer build is clean.

82 Closing perspective

Advanced pointer programming is not about memorizing obscure syntax. It is about developing immediate intuition for three things:

1. where an object is,
2. how long it exists,
3. who is responsible for destroying it.

Once those questions become automatic, raw pointers, smart pointers, DMA buffers, linked structures, network buffers, allocators, and low-level C APIs all become variations of the same underlying problem: representing access to memory while preserving correct lifetime and ownership.